The major advantage of embedded systems is that they are special-purpose; possibly performing only a single function. This function is likely to be required in a wide variety of different manufactured products. Mass production of an embedded system brings economies of scale: the more we make, the cheaper they become. During the early years of their use, embedded systems had the disadvantage that programming was difficult because the memory space available to store a program was limited. For this reason, programs had to be short. In addition, there was the disadvantage that if errors were found following installation then new chips had to be manufactured and used to replace the faulty ones. In modern systems these problems are less likely, but a new problem has developed. Embedded systems are now part of what is called the IoT (Internet of Things). More and more embedded systems are being installed with a network connection. This can greatly improve the usefulness of a product, for example by providing information and updates to the owner. However, this accessibility via a network is a security concern. Embedded systems are less likely to be protected against unlawful actions than general-purpose systems.

**Discussion Point:**

How might useful information from an embedded system installed in a domestic appliance be communicated over a network to the owner of the appliance?

## 3.03 Memory components



KEY TERMS

**Random-access memory (RAM):** volatile memory that can be read from or written to any number of times

**Read-only memory (ROM):** non-volatile memory that cannot be written to but can be read from any number of times



The components that make up the main memory of a general-purpose computer system are called **random-access memory (RAM)**. It is called RAM because any byte of data stored can be accessed without affecting the other bytes stored. Because of this it might have been better called 'direct-access memory'. Another possible name would be 'read–write memory' because RAM can be repeatedly read from or written to. A key feature of RAM is that it is volatile, which means that when the computer system is switched off the contents of the memory are lost.

There are two general types of RAM technology. Dynamic RAM (DRAM) is constructed from capacitors that leak electricity and therefore need regularly recharging (every few milliseconds) to maintain the identity of the data stored. Static RAM (SRAM) is constructed from flip-flops that continue to store data indefinitely while the computer system is switched on. The circuits and logic for flip-flops are discussed in Chapter 19 (Section 19.02)).

The major difference between the two types of RAM is that DRAM requires fewer electronic components per bit stored. This means DRAM is cheaper to make and has a higher density for data storage. The major advantage of SRAM is that it provides shorter access time. In a general-purpose computer system, it is normal practice for main memory to be constructed from DRAM but for cache memory to be provided by SRAM because of the faster access speed. By contrast, embedded systems that need RAM with only limited capacity often use SRAM for this.

The second category of memory component is called **read-only memory (ROM)**. Again, this name does not give a full picture of the characteristics of this type of component. ROM shares the random-access or direct-access properties of RAM. However, as the name implies it

cannot be written to when in use within the computer system. The other key feature is that the data in ROM is not lost when the computer system is switched off; the memory is non-volatile.



**TIP**

The word volatile has several meanings. Try to remember that volatile memory no longer stores data when the system is switched off.

ROM has specialised uses for the storage of data or programs that are going to be used unchanged over and over again. In a general-purpose system the most important use is in storing the bootstrap program. This is a program that runs immediately when a system is switched on. There are a number of other uses for ROM in such a system, some of which we will see later in this book. In addition, ROM is used in many embedded systems.

There are four different types of ROM.

1. In the simplest type of ROM the programs or data are installed as part of the manufacturing process. If different contents are needed the chip must be replaced.
2. An alternative is Programmable ROM (PROM). The manufacturer of the chip supplies chips to a system builder. The system builder installs the program or data into the chips. This allows the system builder to test some samples of programmed chip before committing the whole batch to be programmed. As with the simplest type of ROM, the program or data once installed cannot be changed.
3. A more flexible type of ROM is Erasable PROM (EPROM). The installed data or program can be erased (using ultraviolet light) and new data or a new program can be installed. However, this reprogramming usually requires the chip to be removed from the circuit.
4. The most flexible type of ROM is Electrically Erasable PROM (EEPROM). As the name suggests, this works in a similar way to EPROM, except an electrical signal can be used to remove existing data. This has the major advantage that the chip can remain in the circuit while the contents are changed. However, the chip is still used as read-only.



**Discussion Point:**

Can you find out what memory components are in the computer system you are using and any details about them such as the type and storage capacity?

## Buffers

Whenever data has to be transferred from one part of a computer system to another, a problem occurs if the data can be sent more quickly than it can be received. The solution to the problem is to use a **buffer**. Data enters a buffer before being transmitted to its destination. The buffer functions as a queue so the data emerges in the order that it has entered the buffer. Typically, the buffer is created in the computer memory.



**KEY TERM**

**Buffer:** a temporary storage created for data transmitted from one part of the system to another which functions as a queue

**Question 3.01**

Can you think of examples of data transfer that would need a buffer?